Neural Networks 206 (2027) 109824

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

DeCITS: Incomplete time series clustering with deep contrastive learning

Ying Zhong [a], Haiyan Wang [a], Dong Huang [a], Yuan-Kun Xu [a], Chang-Dong Wang [b,c], Huiru Zhou [a,*]

[a] College of Mathematics and Informatics, South China Agricultural University, Guangzhou, 510642, China
[b] School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou, 510006, China
[c] Guangdong Key Laboratory of Information Security Technology, Guangzhou, 510642, China

ARTICLE INFO

Keywords:
Joint training
Contrastive learning
Incomplete time series
Data augmentation
Deep clustering

ABSTRACT

Time series data often contains missing values in real-world applications, making incomplete time series clustering a significant research focus in recent years. However, incorporating imputation and clustering within a model can lead to rapid error propagation from imputed values. Many existing time series clustering methods struggle to reduce the error propagation from imputation to clustering. Additionally, how to learn cluster-friendly representations from unlabeled time series data with temporal dynamics, and how to isolate the representation extracted from the clustering task without considering their interaction, remain challenging issues. In light of this, this study presents a Deep Contrastive Incomplete Time Series Clustering (DeCITS) method, which utilizes a unified framework for learning representations of incomplete time series in a contrastive learning manner. To mitigate error propagation, an adversarial strategy is incorporated to distinguish real and imputed values, encouraging the model to impute more realistic missing data. Consequently, DeCITS obtains cluster-friendly representations and achieves highly competitive clustering performance. Extensive experiments on six real-world datasets and eight UCR datasets demonstrate the superiority of DeCITS over existing state-of-the-art methods.

1. Introduction

Time series plays an important role in various fields such as financial markets forecasting (Barazandeh et al., 2025), gene expression analysis (Bar et al., 2022), and climate modeling (Kashinath et al., 2021). Due to the absence of category labels, data analysts increasingly employ clustering methods to discover meaningful patterns and valuable information from complex and large-scale time series datasets (Alahamade et al., 2022; Guo et al., 2025; Huang & Deng, 2023).

Real-world time series data often contain missing values due to issues such as data corruption, making re-recording both costly and impractical (Li et al., 2024; Zhu et al., 2021). Incomplete time series can hinder data analysis and negatively impact downstream applications, such as forecasting (Lai et al., 2025) and clustering (Cheema, 2014). For instance, de Jong et al. (2019) successfully stratified patients with Alzheimers disease and Parkinsons disease into subgroups, yet the study did not adequately address clustering multivariate short time series with many missing values. A natural approach to tackle this issue is to first impute the missing values and then perform clustering (Du et al., 2023). However, two-stage strategies often struggle to effectively combine imputation and clustering methods, which may lead to sub-optimal results (Zhang et al., 2025). Recently, attempts have been made to improve this situation. de Jong et al. (2019) proposed a deep learning-based method called Variational Deep Embedding with Recurrence (VaDER), which relies on a Gaussian mixture variational autoencoder framework to produce well-separated representations. A substantial amount of data was used for training. While verified to be effective, the method still exhibits limitations, particularly concerning significant error propagation from imputation to clustering. To address this shortcoming, Ma et al. (2021) introduced an adversarial strategy designed to effectively minimize error propagation during the transition from imputation to clustering.

Contrastive learning has recently demonstrated a robust ability to learn representations across various tasks by applying multiple augmentations to the original data, thereby deriving latent embeddings from the augmented inputs (Chen & He, 2021; Deng et al., 2025). This approach explores different views of input images in the vision domain through data augmentation techniques, maximizing the similarity between views of the same image while minimizing the similarity between views from different images (Li et al., 2026). Additionally, Eldele et al. (2023) enabled the model to learn discriminative representations in a challenging time series prediction task by introducing a temporal contrasting module. Gao et al. (2023) utilized multivariate time series for arbitrary-step

* Corresponding author.
E-mail addresses: yingzhonghere@hotmail.com (Y. Zhong), cshywang@scau.edu.cn (H. Wang), huangdonghere@gmail.com (D. Huang), ykxu@stu.scau.edu.cn (Y.-K. Xu), changdongwang@hotmail.com (C.-D. Wang), huiruzhou@scau.edu.cn (H. Zhou).

https://doi.org/10.1016/j.neunet.2026.109824
Received 17 June 2026; Received in revised form 18 October 2026; Accepted 29 October 2026
Available online 2 November 2026
0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

prediction, combining an explainable Tensorized Gated Recurrent Unit (TGRU) with Ordinary Differential Equations (ODE) to facilitate multi-step predictions at arbitrary time points. However, the aforementioned methods primarily focus on prediction tasks and are not broadly applicable to other time-series domains such as clustering. To date, only a few studies have explored contrastive learning for time series clustering. For example, Zhong et al. (2022) employed jitter, a data augmentation strategy, to model two views and implemented two contrastive losses to optimize clusters. Nevertheless, this method is designed for complete time series and is challenging to extend to incomplete time series due to issues such as data collection failures.

To address these issues, we propose the Deep Contrastive Incomplete Time Series Clustering (DeCITS) model. This model employs simple yet effective data augmentation techniques that adapt to any time-series data, generating alternative representations of the input data. Its primary objective is to maximize the similarity among pairs of identical time series while minimizing the similarity among pairs of distinct time series. Furthermore, we incorporate a secondary component that integrates soft K-means objectives (Ma et al., 2019) within an encoder-decoder network for projecting discrete representations to enhance clustering performance. This approach enables DeCITS to jointly optimize the imputation and clustering processes, thereby reducing the bias introduced by imputation and ensuring that the imputed values are more conducive to effective clustering.

The primary contributions of this work are as follows:

- Missing values break temporal dependencies in incomplete time series clustering. Existing one-stage methods suffer from exploding bias as imputation errors propagate uncontrollably into clustering. We introduce an adversarial strategy that suppresses bias amplification and cuts off error propagation, breaking the vicious cycle between imputation and clustering.
- Conventional contrastive learning fails under missing data because corrupted similarity measures yield more false positive/negative pairs. We apply a straightforward yet effective augmentation technique for incomplete time series and perform cluster-level contrastive learning by contrasting cluster assignments instead of raw instances, making the objective robust to missing values while enhancing clustering structure.
- To stabilize one-stage joint optimization, we insert a fully connected layer between imputation and clustering as a learnable buffer that decouples the two tasks. Imputation and clustering are optimized simultaneously to ensure imputation serves clustering and to mitigate optimization instability.
- The proposed mechanisms were validated on six real-world and eight synthetic UCR incomplete time series datasets. Results consistently show that our method resolves the core contradiction of incomplete time series clustering and achieves state-of-the-art performance.

2. Related work

This section first reviews the incomplete time series clustering models, and then introduces contrastive learning for time series methods.

2.1. Incomplete time series clustering

Early studies primarily focused on clustering complete time series using unsupervised learning (Liao, 2005). Recently, there has been an emergence of studies exploring modeling based on incomplete time series variants. The existing literature on incomplete time series clustering methods can be broadly classified into two groups: two-stage methods involving imputation followed by clustering, and one-stage methods involving joint optimization.

Two-stage methods initially impute missing values before applying existing clustering techniques to the complete time series data (Chen et al., 2022). Common interpolation methods, such as zero, mean, or median imputation, typically replace missing values using nearby observed values, thereby neglecting any long-term dependencies inherent in the time series (Hussein et al., 2025). Recently, complex algorithms based on deep learning have gained prominence for filling in missing values. Cao et al. (2018) introduced Bidirectional Recurrent Imputation for Time Series (BRITS), which recovers missing values without specific assumptions about the underlying data generation process, thereby enhancing the generality of the model. The Generative Adversarial Imputation Nets (GAIN) (Yoon et al., 2018) framework and Multivariate Time Series Imputation with Generative Adversarial Networks (MisGAN) (Luo et al., 2018) have also been employed to impute missing values. Fortuin et al. (2020) proposed GP-VAE, which combines a Gaussian process prior with a variational autoencoder for multivariate time series imputation, providing not only accurate imputations but also interpretable uncertainty estimates. Similarly, Zhang et al. (2023) proposed TABiG, which employs tri-attention and bidirectional GRU to capture temporal and inter-feature dependencies, along with a realistic masking strategy for enhanced generalization. Although these algorithms have achieved considerable success in imputation, they are still utilized as a data preprocessing step for clustering tasks. Optimizing the best combination of imputation and clustering processes is crucial for enhancing overall clustering performance.

One-stage methods conduct clustering analysis directly on incomplete time series data while accounting for the effects of missing values (Li et al., 2024). These approaches integrate imputation and clustering into a unified process, aiming to preserve the characteristic information of the original data as much as possible. Although the one-stage method can better accommodate incomplete data, it necessitates the development of suitable optimization strategies and algorithms for effective joint optimization. For instance, Ma et al. (2021) introduced Clustering Representation Learning on Incomplete Time Series (CRLI), an unsupervised temporal representation learning model that simultaneously optimizes the imputation and clustering processes to impute more distinctive values for clustering, resulting in well-defined clustering properties in the learned representations. Moreover, the inclusion of a discriminator helps align the distribution of the imputation values with the true distribution, thereby minimizing error propagation from imputation to clustering. Additionally, Mikalsen et al. (2018) proposed Time Series Cluster Kernel (TCK), which learns similarities among incomplete multivariate time series via an ensemble of Gaussian mixture models without explicit imputation. TCK effectively bypasses the bias introduced by explicit imputation, and it relies on traditional kernel-based similarity measures and does not leverage deep representation learning.

Despite the above advances, existing one-stage methods still face two critical challenges. First, they often lack effective mechanisms to explicitly control the quality of imputed values during joint optimization, which may allow imputation errors to propagate into clustering. Second, the learned representations are not explicitly encouraged to be cluster-friendly in a discriminative manner. In contrast, our proposed DeCITS addresses these limitations by incorporating: (1) an adversarial imputation module that distinguishes real from imputed values to ensure imputation fidelity, (2) an instance-level contrastive learning strategy that explicitly shapes the representation space to be more separable, and (3) a joint optimization framework with task-decoupling fully connected layers that stabilizes training and mitigates error propagation from imputation to clustering.

2.2. Contrastive learning for time series

Contrastive learning for time series analysis is gaining significant popularity. For instance, Hu et al. (2023) introduced a time series forecasting model utilizing an enhanced cycle generative adversarial data augmentation method to generate samples closely resembling real data, thereby minimizing contrastive loss. Similarly, Liu et al. (2023) proposed the Attentional Contrastive Calibrated Transformer (ACCT) for diagnosing samples under transient speed conditions. They developed a

regularization approach based on unsupervised contrastive learning to calibrate attention distribution, resulting in a more discriminative feature representation. In the context of multivariate time series classification, Hao et al. (2023) proposed a novel mixed supervised contrastive learning framework that leverages spatial and temporal channels to extract complex spatio-temporal features, while also introducing a mixed supervised contrastive loss to capture multi-scale contextual information and enhance representation learning. Nevertheless, the aforementioned methods largely overlook the fact that real-world time series data are frequently affected by missing values at varying frequencies. To address the temporal dynamics of incomplete time series, Peng et al. (2023) proposed the Generative Contrastive Attentive Spatial-Temporal Network (GCASTN), which integrates generative principles into contrastive self-supervised learning to establish a new paradigm for imputation without reliance on the ground truth of missing data. GCASTN can adaptively learn the reliable spatial-temporal dependencies within incomplete traffic data. Most recently, Tan et al. (2026) proposed the Evolving-masked MTS Clustering (EMTC) for multivariate time series clustering, which introduces an importance-aware masking module to suppress redundant timestamps and employs cluster-guided contrastive learning to enhance cluster separation.

Distinct from all the above contrastive learning methods, DeCITS is specifically designed for incomplete time series clustering. It uniquely combines instance-level contrastive learning with adversarial imputation and joint optimization, forming a unified framework that simultaneously (1) imputes missing values with high fidelity, (2) learns cluster-friendly representations, and (3) mitigates error propagation from imputation to clustering. To the best of our knowledge, DeCITS is the first approach to integrate these three components within a contrastive learning paradigm for incomplete time series clustering.

3. Method

3.1. Framework overview

This section elaborates on our proposed DeCITS. As illustrated in Fig. 1, our model comprises four interconnected components: a parallel backbone with an imputation module, an adversarial module, a reconstruction module, and a contrastive module. A parallel backbone with an imputation module is introduced to delve into the latent temporal representations of the data using these encoders. In one branch of this module, a distinct perspective of the input data is generated using a simple yet effective augmentation technique called jitter. Then, two encoders independently impute the missing values, resulting in two sets of imputed data. Within the adversarial module, the discriminator treats the imputed time series data as input and calculates the probability of observation for each variable at every time step. In the reconstruction module, the last hidden states of the encoder are concatenated to extract stable latent representations, followed by passing them through a fully-connected layer. Subsequently, a K-means objective is incorporated into the encoder-decoder network to reconstruct the input data. The contrastive module focuses on enhancing intra-sample view similarity and reducing inter-sample view similarity. The subsequent subsections will provide a detailed overview of each module.

3.2. Parallel backbone with imputation

Data augmentation is a crucial component contributing to the success of contrastive learning methods (Sun et al., 2026). These methods seek to maximize similarity among different sequences of the same sample while minimizing similarity among sequences from different samples. Therefore, it is essential to develop appropriate time-series data augmentations for contrastive learning. In this module, we utilize a jitter strategy, which simulates additive sensor noise. Importantly, the hyperparameters for augmentation must be selected carefully, considering the characteristics of the time-series data. For instance, the ratio for unnormalized data should be significantly higher than the jitter ratio for normalized time-series data. Subsequently, the raw inputs and their corresponding randomly jittered data are concurrently imputed and then fed separately into the encoders to obtain latent representations.

Considering a multivariate input time series dataset: $X_{data} = \{X_1; \dots; X_i; \dots; X_N\}$, where $X_i = (x_1, \dots, x_t, \dots, x_T) \in \mathbb{R}^{T\times K}$ denotes each time series, $x_t \in \mathbb{R}^K$, N denotes a mini-batch size of training set, and K denotes the K-dimensional input at each time step t. Suppose a time series dataset X_{data} has missing values which can be indicated by a corresponding indicator set $M = \{M_1; \dots; M_i; \dots; M_N\}$, where each T-dimensional mask indicator M_i is defined as $M_i = (m_1, \dots, m_t, \dots, m_T)$ and $m_t \in \{0, 1\} \in \mathbb{R}^K$. The corresponding element of m_t is 1 if the element of x_t is completed, and 0 if the element of x_t is missing.

For a mini-batch of incomplete time series samples, we denote theirs augmented version as $X^a_{data} = \{X^a_1; \dots; X^a_i; \dots; X^a_N\}$. Each time series X^a_i is defined as $X^a_i = (x^a_1, \dots, x^a_t, \dots, x^a_T)$, where $X^a_i \in \mathbb{R}^{T\times K}$ and $x^a_t \in \mathbb{R}^K$. In our augmented case, there is also a corresponding indicator set $M^a = \{M^a_1; \dots; M^a_i; \dots; M^a_N\}$, where each indicator M^a_i is represented as $M^a_i = (m^a_1, \dots, m^a_t, \dots, m^a_T)$ and $m^a_t \in \{0, 1\} \in \mathbb{R}^K$. When the component of x^a_t contains value, we set the corresponding component of m^a_t to 1 and vice versa.

The framework of our DeCITS is illustrated in Fig. 1. In the upper branch, the encoder employs a modified Recurrent Neural Network (RNN) to capture multi-scale and dynamic time series, as the x_t with missing values cannot be directly input into a standard RNN. Therefore, the network is trained to predict the next input variable's value, utilizing the current predicted value when the input is incomplete. When the next input variable is complete, it will be used as the target value. In this paper, we train $\hat{x}_t$ to approximate the next vector x_t when it is completed using the last hidden states h_{t-1} as follows:

$$\hat{x}_t = W_{imp} h_{t-1} + b_{imp}, \tag{1}$$

where W_{imp} is a learnable parameter and b_{imp} is a bias term. Due to $\hat{x}_t$ is trained to approximate the next vector x_t, it can be employed to impute it when x_t containing missing values. Thus, the imputed value u_t is computed as follows:

$$u_t = m_t \odot x_t + (1 - m_t) \odot \hat{x}_t, \tag{2}$$

where $\odot$ is the element-wise product.

The revised RNN is trained using the completed input vector u_t, the updated equation of the encoder can be written as follows:

$$h_t = \tanh\left(W_h h_{t-1} + W_x u_t + b\right), \tag{3}$$

where h_t denotes the hidden unit vector at timestamp t, W_x and W_h represent the input-to-hidden and hidden-to-hidden parameters, respectively. At the same time, on the lower branch, the update equation of an identical revised RNN for augmented time series can be summarized as follows:

$$\hat{x}^a_t = W^a_{imp} h^a_{t-1} + b^a_{imp}, \tag{4}$$

$$u^a_t = m^a_t \odot x^a_t + (1 - m^a_t) \odot \hat{x}^a_t, \tag{5}$$

$$h^a_t = \tanh\left(W^a_h h^a_{t-1} + W^a_x u^a_t + b^a\right), \tag{6}$$

where W^a_{imp} and b^a_{imp} are also learnable parameters for augmented data and Eqs. (5) replaces the missing values of x^a_t by the corresponding elements of predict results $\hat{x}^a_t$. Eqs. (6) is calculated the hidden state for augmented version at time step t based upon the imputed vector u^a_t.

When the entire incomplete input sequence and its augmented version individually fed into the network, we can obtain the corresponding predicted sequence $\hat{X}$ and $\hat{X}^a$ by Eqs. (2) and (4) , which represented as

$$\hat{X} = \left\{\hat{x}_1, \hat{x}_2, \dots, \hat{x}_T\right\}, \tag{7}$$

$$\hat{X}^a = \left\{\hat{x}^a_1, \hat{x}^a_2, \dots, \hat{x}^a_T\right\}. \tag{8}$$

Consider the issue where the first value of time sequence may be missing, we solve the problem by introducing an identifier as the starting

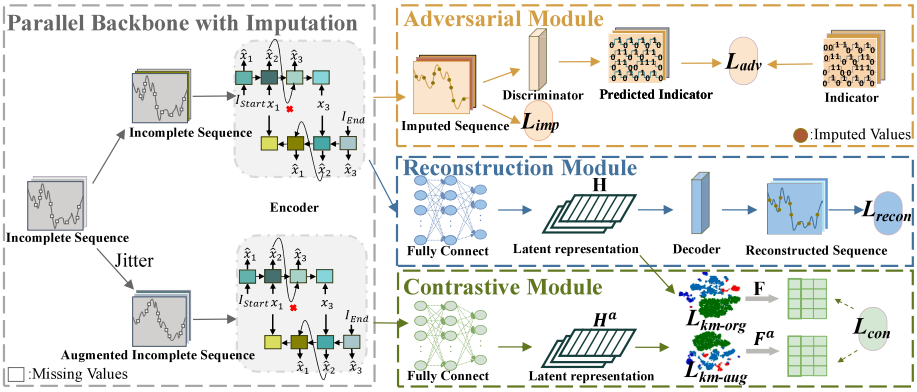


Fig. 1. The overall architecture of the proposed DeCITS method.

symbol of the RNN. Hence, the identifiers for two different branches are respectively defined as:

$$S = cW_s, \tag{9}$$

$$S^a = c^a W_s^a, \tag{10}$$

where $c \in \mathbb{R}^1$ and $c^a \in \mathbb{R}^1$ are the constant values and they are respectively transfered to the same dimension of x_t and x_t^a by using adaptable weight matrices $W_s \in \mathbb{R}^{K\times 1}$ and $W_s^a \in \mathbb{R}^{K\times 1}$. Here, K denotes the input dimension per time step. Moreover, in order to reduce the errors of imputed values, we use the encoders instantiated by two identical bidirectional RNNs. We utilize $S_{forward}$ for the forward RNN and $S_{backward}$ for the backward RNN (reverse the input sequence and then utilize it as input) to enable the encoders distinguish the different directions.

3.3. Three levels of learning

In this section, three levels of learning are introduced in detail, optimizing the whole model jointly.

3.3.1. Adversarial strategy

For the imputation task, we denote the predicted sequence $\hat{X}_i = \{\hat{x}_1, \hat{x}_2, \ldots, \hat{x}_T\}$ of the ith time series sample, the imputation loss of a mini-batch size of training time series samples is computed as follows:

$$\mathcal{L}_{imp}(X, \hat{X}, M) = \frac{1}{N}\sum_{i=1}^{N} \|(X_i - \hat{X}_i) \odot M_i\|_2^2, \tag{11}$$

where Eq. (11) denotes the mean squared error loss between the input sequence and the predict sequence. At decoding step t, observed coordinates use the ground-truth input (where $m_t = 1$), while missing coordinates use the previous prediction (where $m_t = 0$). Thus $\mathcal{L}_{imp}$ supervises the predictive path on observed entries and stabilizes the autoregressive imputation used to form the completed sequence.

However, the network of DeCITS inevitably introduces error propagation from imputation to clustering since the imputed values may lead to imputation bias. This problem can be quickly amplified as it is fed into the encoder, resulting in the exploding bias error and further hindering clustering performance. Hence, we introduce a discriminator D to alleviate the negative impact of missing values. Specially, for the ith time series training sample $U_i \in \mathbb{R}^{T*K}$, there is a completed sequence $U_i = (u_1, u_2, \ldots, u_T)$ given by Eq. (2) and its corresponding mask indicator sequence $M_i = (m_1, m_2, \ldots, m_T)$. U_i can be regarded as the average of the completed sequence of the forward and backward RNN. For each U_i, we can realize which value in U_i is real value and which value is predicted. We utilize completed time series sample U_i to train a discriminator D by minimizing the discriminator loss and then use it to guide the encoder in turn, so that the distribution of the predicted results is closer to the distribution of the true one. The discriminator loss can be written as follows:

$$\mathcal{L}_D(U, M) = -[\underbrace{E\log(D(X_{real}))}_{X_{real}\sim U} + \underbrace{E\log(1 - D(\hat{X}_{imp}))}_{\hat{X}_{imp}\sim U}] \tag{12}$$

$$= -\frac{1}{N}\sum_{i=1}^{N}[M_i \odot \log(D(U_i)) + (1 - M_i) \odot \log(1 - D(U_i))], \tag{13}$$

where $D(\cdot)$ represents the estimated indicator probability M_i of the discriminator. Note that the discriminator maximizes the log output for real values in the first term of Eqs. (12) while tries to minimize its output for imputed values in the second term of Eqs. (12). The discriminator is consisted of five RNN cells and a fully connected layer, utilizing global contextual information to learn which values are actually-observed and which values are imputed in an imputed sequence.

The DeCITS is managed to make the distribution of imputed values closer to the real ones by training the discriminator D. The adversarial loss of the DeCITS yields:

$$\mathcal{L}_{adv}(U, M) = \frac{1}{N}\sum_{i=1}^{N}(1 - M_i) \odot \log\left(1 - D(U_i)\right). \tag{14}$$

In practice we optimize the generator with a non-saturating adversarial objective implemented as sigmoid cross-entropy on discriminator logits, applied only at missing positions, which encourages imputed entries to be classified as "real" and mitigates vanishing gradients associated with the saturating $\log(1 - D)$ form when D is confident. Finally, we alternately update the parameters of the discriminator and the encoder.

3.3.2. Reconstruction module

This end-to-end framework reads the incomplete time series data and its augmented data. Next, the latent representations extracted by the encoders should contain enough information to form the cluster structure. Therefore, the encoder for taking each origin incomplete time series sample and its missing indicator as input can be defined as:

$$H_i = f_{\text{enc}}(X_i, M_i), \tag{15}$$

where f_{enc} represents the update function that maps the incomplete time series sample and its corresponding indicator into the latent temporal representation without any pre-imputation. For the augmented branch, the encoder can be rewritten as follows:

$$H_i^a = f_{\text{enc}}(X_i^a, M_i^a), \tag{16}$$

Specially, to obtain output H_i (or H_i^a), we concatenate the last hidden state of the forward RNN and backward RNN. Note that it is important for extracting the informative features of original time series. We introduce a decoder to reconstruct the original incomplete time series X_i. However, H is directly fed into the decoder that may lead to the model instability. Hence, we use a fully connected layer, which can improve the stability of DeCITS and further enhance the clustering performance.

The reconstruction loss of a mini-batch size of original time series samples is calculated by using masked Mean Square Error (MSE) as follows:

$$\mathcal{L}_{rec}(U, H, M) = \frac{1}{N}\sum_{i=1}^{N}\left\|\left(U_i - f_{\text{dec}}(H_i)\right)\odot M_i\right\|_2^2. \tag{17}$$

Here $\odot$ denotes element-wise (Hadamard) product. Eq. (17) only reconstructs original time series data corresponding to the observed components, regardless of missing ones.

3.3.3. Contrastive module

Theoretically, $H \in \mathbb{R}^{h*N}$ and $H^a \in \mathbb{R}^{h*N}$ are static matrice, where H and H^a denote h-dimension representations. However, the learned representations H and H^a are fail to achieve clustering task. Therefore, Ma et al. (2019) try to introduce two soft K-means objectives to motivate H and H^a to form clustering structure respectively. The losses $\mathcal{L}_{km-org}$ and $\mathcal{L}_{km-aug}$ are computed within a mini-batch as follows:

$$\mathcal{L}_{km-org} = \text{Tr}\left(H^T H\right) - \text{Tr}\left(F^T H^T H F\right)\text{s.t.} F^T F = I, \tag{18}$$

$$\mathcal{L}_{km-aug} = \text{Tr}\left[(H^a)^T H^a\right] - \text{Tr}\left[(F^a)^T (H^a)^T H^a F^a\right]\text{s.t.}(F^a)^T F^a = I, \tag{19}$$

where $F \in \mathbb{R}^{N*K_c}$ and $F^a \in \mathbb{R}^{N*K_c}$ separately represent the cluster indicator matrices and K_c denotes the number of clusters. Since H is obtained dynamically from the network instead of statically, we train Eq. (18) using an update two-step method.

1. First, fixing F, we update H by using standard stochastic gradient descent (SGD), which guides the representations form cluster structures.
2. Second, fixing H, updating F can follow the KyFan theorem by computing the k-truncated singular value decomposition (SVD) of H.

Note that directly employing Eq. (18) (or Eq. (19)) to optimize H (or H^a) may lead to an instability problem of DeCITS, further making the model fail to boost clustering performance. Thus, to tackle this problem, we propose a fully connected layer to alleviate instability of the Contrastive model. Next, we can derive a clustering contrastive loss to maximize the similarity within positive pairs and minimize the similarity within negative pairs.

For a mini-batch of N input time series, we can obtain the cluster indicators $F \in \mathbb{R}^{N*K_c}$ and $F^a \in \mathbb{R}^{N*K_c}$ for the two parallel branches by calculating the k-truncated SVD of H and H^a, where K_c is the number of clusters and the ith column (i.e., the F of ith cluster, denoted as $\hat{F}_i$ and $\hat{F}_i^a$) of these two matrices indicate the probability of allocating each sample to the ith cluster. The C is the cluster-level representation set which includes the representations of all clusters from two parallel branches, that is, $C = \{\hat{F}_1, \ldots, \hat{F}_{K_c}, \hat{F}_1^a, \ldots, \hat{F}_{K_c}^a\}$. Formally, for a cluster $\hat{F}_i$, we can obtain $2K_c - 1$ cluster pairs , and hence, $\{\hat{F}_i, \hat{F}_i^a\}$ is considered as a positive cluster pair. Meanwhile, the remaining $2K_c - 2$ are the negative cluster pairs of $\hat{F}_i$. Then, we use cosine distance to measure the similarity between the cluster representations, the cluster-level contrastive loss for $\hat{F}_i$ is denoted,

$$\hat{l}_i = -\log\frac{\exp\left(s\left(\hat{F}_i, F^+\right)/\tau\right)}{\sum_{F^-\in C}\exp\left(s\left(\hat{F}_i, F^-\right)/\tau\right)}, \tag{20}$$

where τ is a temperature parameter. F^+ and F^- are the positive and negative clusters for $\hat{F}_i$, respectively. Then the overall cluster-level contrastive loss is

$$\mathcal{L}_{con} = \frac{1}{2K}\sum_{i=1}^{K}(\ell_i + \ell_i^a) - H(F), \tag{21}$$

To avoid the trivial solution that assigns most cluster-level representations to the same cluster, we introduce the entropy term $H(F)$, i.e.,

$$H(F) = -\sum_{i=1}^{K}\left[P\left(\hat{F}_i\right)\log P\left(\hat{F}_i\right) + P\left(\hat{F}_i^a\right)\log P\left(\hat{F}_i^a\right)\right], \tag{22}$$

where $P\left(\hat{F}_i\right) = \frac{\sum_{j=1}^{N} f_{ji}}{\left\|\hat{F}_i\right\|_1}$, $P\left(\hat{F}_i^a\right) = \frac{\sum_{j=1}^{N} f_{ji}^a}{\left\|\hat{F}_i^a\right\|_1}$ denote cluster assignment probabilities. f_{ji} (or f_{ji}^a) defines the entry in the jth row and ith column of the cluster-level representation matrix F (or F^a, respectively).

3.4. The overall loss of DeCITS

With the above definitions, the overall training loss of DeCITS can be defined as

$$\mathcal{L}_{DeCITS} = \mathcal{L}_{\text{imp}} + \mathcal{L}_{\text{rec}} + \mathcal{L}_{adv} + \lambda * (\mathcal{L}_{km-org} + \mathcal{L}_{km-aug}) + \mathcal{L}_{con}, \tag{23}$$

where λ is coefficient denoting the relative weight of k-means losses. The Eq. (23) is minimized to learn the cluster-friendly representations on incomplete time series. $\mathcal{L}_{\text{imp}}$ (Eq. (11))aims to capture the temporal dynamics of the incomplete time series. $\mathcal{L}_{\text{rec}}$ (Eq. (17))makes the representations reconstruct the original time series input. $\mathcal{L}_{adv}$(Eq. (14)) reduces the error propagation from imputation to clustering process, further improving the ability of the model. $\mathcal{L}_{km-org}$ (Eq. (18))and $\mathcal{L}_{km-aug}$(Eq. (19)) stimulate the representations to form cluster structure. $\mathcal{L}_{con}$(Eq. (21)) aims to optimize the model, further forming better clustering results.

4. Experiments

4.1. Time series datasets and evaluation indicators

In this study, we conduct experiments on 14 datasets, including six real-world incomplete time series datasets sourced from various domains (Alizadeh et al., 2000; Bianchi et al., 2017; Chen et al., 2002; Silva et al., 2012) and eight UCR time series datasets (Mejri et al., 2024; Wu & Keogh, 2023). Among them, 10 representative datasets are adopted for the ablation study and the main clustering performance comparison to ensure a fair and concise evaluation against all baselines. In the imputation performance experiments, both Mean Absolute Error (MAE) and Root Mean Square Error (RMSE) are reported for all eight UCR datasets. In addition, four UCR datasets (Cricket_X, Cricket_Y, DiatomSizeReduction, and Meat) are reserved for stability and sensitivity analyses to evaluate the robustness of DeCITS under various missing mechanisms, with and without fully-connected layers, and at different missing ratios. The details of these 14 datasets are presented in Table 1.

Unless otherwise specified, incomplete data for the UCR datasets are simulated under the Missing Completely At Random (MCAR) mechanism with a fixed random seed (seed = 42) and a missing ratio of 20%. This missing ratio is consistent with the commonly adopted settings in representative imputation studies such as BRITS (Cao et al., 2018) and GAIN (Yoon et al., 2018), which facilitates fair comparison with existing methods. Each dataset follows the official train/test split, with training sets used for model training and testing sets for evaluation. Table 1 shows the number of training and test instances for each dataset.

We assess clustering performance using the Rand Index (RI) and Normalized Mutual Information (NMI). The NMI is calculated as follows (Strehl & Ghosh, 2002):

$$\text{NMI} = \frac{\sum_{i,j=1}^{K} N_{i,j}\log\left(\frac{N\cdot N_{i,j}}{t_i p_j}\right)}{\sqrt{\left(\sum_{i=1}^{K} t_i \log\left(\frac{t_i}{N}\right)\right)\left(\sum_{j=1}^{K} p_j \log\left(\frac{p_j}{N}\right)\right)}}, \tag{24}$$

where N denotes the number of time series, p_j and t_i are the numbers of the data in the cluster K_j and in the ith true label. $N_{i,j}$ denotes the number of instances belonging to the intersection between the cluster K_j and the ith true label. For this evaluation metric, the bigger value, the better clustering result.

The RI is calculated as follows (Rand, 1971):

$$\text{RI} = \frac{\text{TP} + \text{TN}}{N(N-1)/2}, \tag{25}$$

where true positive (TP) is the number of pairs of time series that are correctly belong to the same cluster, true negative (TN) is the number of pairs that are correctly belong to different clusters.

Table 1
Summary of 14 datasets used in the study.

Number	DATASETS	Train/Test	Dim	Missing ratio	Length	Classes
1	alizadeh-2000-v1-filter[a]	29/13	1	1.84	932	2
2	alizadeh-2000-v3-filter[a]	43/19	1	2.59	1032	4
3	BloodSample[a]	707/176	10	85.72	20	2
4	chen-2002-filter[a]	125/54	1	2.31	2328	2
5	HouseVote[a]	303/131	1	5.41	16	2
6	Physionet 2012[a]	2798/1199	35	80.52	48	2
7	Adiac	390/391	1	20	176	37
8	Beef	30/30	1	20	470	5
9	BeetleFly	20/20	1	20	512	2
10	CBF	30/900	1	20	128	3
11	Cricket_X	390/390	1	20	300	12
12	Cricket_Y	390/390	1	20	300	12
13	DiatomSizeReduction	16/306	1	20	345	4
14	Meat	60/60	1	20	448	3

[a] denotes real-world time series datasets.

4.2. Baseline methods

The proposed DeCITS is compared with two state-of-the-art (SOTA) one-stage methods and several two-stage methods. The two-stage methods first impute missing values using two SOTA imputation techniques, namely BRITS (Cao et al., 2018), GAIN (Yoon et al., 2018), as well as one representative zero imputation method (ZERO) (Ma et al., 2021). These imputation methods are briefly summarized as follows.

1. **ZERO** (Ma et al., 2021): The method directly replace all the missing value with zero.
2. **BRITS** (Cao et al., 2018): The Bidirectional Recurrent Imputation for Time Series (BRITS) model that jointly trains imputation and classification and the category information is proposed to get better imputed value.
3. **GAIN** (Yoon et al., 2018): The Generative Adversarial Imputation Nets (GAIN) model that introduces a hint mechanism to train the correct generator.

Then we introduce existing SOTA clustering methods including DEC (Xie et al., 2016), IDEC (Guo et al., 2017), DTC (Sai Madiraju et al., 2018), DTCR (Ma et al., 2019). We briefly summarize these deep clustering methods as follows.

1. **DEC** (Xie et al., 2016): The unsupervised Deep Embedded Clustering (DEC) model simultaneously learns a mapping from the data space to a lower-dimensional feature representations in which it iteratively optimizes a clustering objective.
2. **IDEC** (Guo et al., 2017): The Improved Deep Embedded Clustering (IDEC) model with local structure preservation utilizes feature space to scatter data via optimizing a Kullback-Leibler divergence-based clustering loss and maintains the local structure carefully.
3. **DTC** (Sai Madiraju et al., 2018): A Deep Temporal Clustering (DTC) model takes the Kullback-Leibler (KL) divergence between the clustering objective and the dimensionality reduction objective as guidance to train non-linear features in deep framework.
4. **DTCR** (Ma et al., 2019): The Deep Temporal Clustering Representation (DTCR) manipulates a fake sample strategy to improve ability of the temporal encoder and guides representation learning by a novel k-means objective.

Finally, the SOTA one-stage methods are specifically described as follows.

1. **VaDER** (de Jong et al., 2019): The Variational Deep Embedding with Recurrence (VaDER), a deep learning model for clustering of multivariate clinical patient trajectories with missing values, realizes the variational auto-encoder clustering framework into RNN and combines imputation with training process to directly cluster incomplete time series data.
2. **CRLI** (Ma et al., 2021): The Clustering Representation Learning on Incomplete time-series data (CRLI) model jointly optimizes the imputation and clustering process to impute more discriminative values for clustering and make the learned representations possessed good clustering property.

4.3. Parameter setting

An adapted bidirectional multi-layer recurrent neural network (RNN) with Gated Recurrent Units (GRU) (Cho et al., 2014) serves as the encoder, while a single-layer RNN with GRU units functions as the decoder. The number of layers in the encoder, denoted as l, is selected from the set 1,2, and the number of units in each layer of the encoder, denoted as h, is chosen from 50,100. The last hidden states of the encoder are concatenated and subsequently input into a fully connected layer to learn stable representations. The decoder utilizes the obtained representation as its initial state, with the forward identifier $S_{forward}$ serving as the starting drive signal for iterative prediction. Thus, the output at time $t-1$ is input at time t. The discriminator consists of a five-layer RNN, with each layer containing the following number of units: 32, 16, 8, 16, and 32. The Gaussian jitter standard deviation ($\sigma = 0.03$) is used for time-series augmentation. The parameters for the comparison methods are established according to the descriptions provided in their original papers. The parameter λ is fixed at 1e-3, and the batch size is set to 32. All models are trained for 200 epochs using the training set and then evaluated on the test set. The number of clusters K_c is set to the ground-truth class number for each dataset, which is only used for evaluation and does not involve test-label leakage during training. The experiments are executed on the TensorFlow platform using an NVIDIA GeForce RTX 3090 GPU and a CPU with 12 cores operating at 2.6 GHz. The mask generation code, the source code and pre-trained models will be released upon publication at the GitHub repository.

4.4. Comparison experiments

In this section, we assess the performance of DeCITS in comparison with previous representative clustering methods across various datasets. Tables 2 and 3 present the results of DeCITS and other representative algorithms on time series datasets, measured in terms of NMI and RI, respectively. The results in Tables 2 and 3 indicate that DeCITS achieves significant performance improvements on most datasets, highlighting the effectiveness of deep clustering in capturing clustering information.

As illustrated in Table 2, DeCITS achieves the best results on 8 out of 10 datasets, representing the highest performance among all evaluated methods. For the RI metric, DeCITS secures the top position on 8 datasets. Additionally, DeCITS attains the highest NMI score of 0.83 on the Ali-v3 dataset and the highest RI score of 0.95 on the Adiac dataset. To conduct a comprehensive comparison between DeCITS and the other

Table 2
Normalized mutual information (NMI) comparison on the 10 incomplete time series datasets.

DATASETS	ZERO				GAIN				BRITS				VaDER	CRLI	Ours[b]
	DEC	IDEC	DTC	DTCR	DEC	IDEC	DTC	DTCR	DEC	IDEC	DTC	DTCR			
Ali-v1	**[0.68]**	0.38	0.02	**[0.68]**	0.50	0.23	0.20	0.25	**[0.68]**	0.64	0.01	**0.65**	0.41	**[0.68]**	**[0.68 ± 0.0000]**
Ali-v3	0.68	0.66	0.56	0.29	0.23	0.46	0.09	0.48	**0.72**	0.55	0.51	0.45	0.23	0.70	**[0.83 ± 0.0512]**
BloodSample	0.21	0.25	0.29	0.04	0.20	0.20	0.00	0.03	0.27	0.29	0.39	0.06	0.20	**0.44**	**[0.53 ± 0.0115]**
Chen	0.05	0.04	0.15	0.02	**0.44**	0.06	0.01	0.06	0.25	0.19	0.16	0.01	0.01	0.21	**[0.45 ± 0.0722]**
Vote	0.64	**0.66**	0.12	0.00	0.60	0.54	0.00	0.01	0.60	**0.66**	0.35	0.42	0.14	0.62	**[0.75 ± 0.0295]**
Physionet	0.01	0.01	0.00	0.00	0.00	0.00	0.00	0.01	**0.02**	**[0.03]**	0.01	0.00	**[0.03]**	0.01	**[0.03 ± 0.0043]**
Adiac	0.42	0.42	0.50	0.41	0.41	0.48	0.57	0.49	0.31	0.20	0.44	0.40	0.41	**[0.62]**	**0.60 ± 0.0145**
Beef	0.37	0.34	0.28	0.38	0.34	0.35	0.28	**[0.51]**	0.31	0.31	0.31	**0.49**	0.34	0.39	**0.49 ± 0.0069**
BeetleFly	0.04	0.07	0.10	0.22	0.03	0.03	0.01	0.16	0.00	0.00	0.00	0.03	**0.31**	**[0.35]**	**[0.35 ± 0.0000]**
CBF	0.26	0.30	0.26	0.31	0.27	0.28	0.28	0.61	0.12	0.14	0.15	0.24	0.42	**0.63**	**[0.67 ± 0.0433]**
AVG Rank	6.40	6.90	8.7	8.6	8.4	8.6	11.6	7.4	7.5	7.9	9.1	9.7	8.1	2.8	**1.2**
p-value[a]	0.0077	0.0020	0.0020	0.0077	0.0020	0.0020	0.0020	0.0059	0.0077	0.0077	0.0020	0.0077	0.0077	0.0249	

The best results are highlighted in bold and bracket and the suboptimal results are highlighted in bold.
[a] p-value < 0.05 indicating that the NMI of this algorithm is significantly lower than that of DeCITS.
[b] The mean±standard deviation are reported from 5 repeated runs with different random seeds.

Table 3
Rand index (RI) comparison on the 10 incomplete time series datasets.

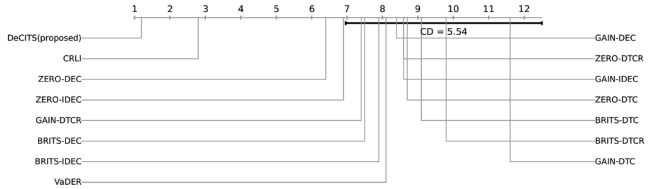
DATASETS	ZERO				GAIN				BRITS				VaDER	CRLI	Ours[b]
	DEC	IDEC	DTC	DTCR	DEC	IDEC	DTC	DTCR	DEC	IDEC	DTC	DTCR			
Ali-v1	**[0.85]**	0.72	0.47	**[0.85]**	0.46	0.62	0.49	0.56	**[0.85]**	**0.82**	0.47	**0.82**	0.47	**[0.85]**	**[0.85 ± 0.0000]**
Ali-v3	0.77	0.77	0.73	0.63	0.51	0.64	0.48	0.72	0.81	0.74	0.70	0.69	0.29	**0.85**	**[0.91 ± 0.0234]**
BloodSample	0.67	0.67	0.72	0.50	0.67	0.65	0.55	0.51	0.62	0.64	0.75	0.59	0.67	**0.81**	**[0.85 ± 0.0085]**
Chen	0.51	0.50	0.56	0.51	**[0.77]**	0.49	0.50	0.51	0.65	0.61	0.54	0.50	0.50	0.58	**0.75 ± 0.0223**
Vote	0.83	0.84	0.57	0.50	0.81	0.76	0.63	0.50	0.81	**0.87**	0.69	0.76	0.58	0.82	**[0.90 ± 0.0167]**
Physionet	0.52	0.52	0.50	0.64	0.51	0.51	0.66	**0.73**	0.51	0.50	0.50	0.52	0.70	0.52	**[0.76 ± 0.0000]**
Adiac	**[0.95]**	**[0.95]**	**0.94**	**[0.95]**	0.91	0.88	0.93	**0.94**	0.87	0.61	0.93	**0.94**	0.59	**[0.95]**	**[0.95± 0.0019]**
Beef	0.73	0.73	0.60	0.74	0.68	0.69	0.60	**[0.76]**	0.66	0.66	0.62	**0.75**	0.62	0.70	**[0.76 ± 0.0087]**
BeetleFly	0.50	0.52	0.54	0.52	0.49	0.49	0.48	0.56	0.47	0.47	0.48	0.49	**[0.62]**	**0.61**	**0.61± 0.0000**
CBF	0.64	0.65	0.60	0.62	0.65	0.65	0.61	0.78	0.58	0.59	0.58	0.61	0.62	**0.80**	**[0.83 ± 0.0300]**
AVG Rank	4.9	5.3	9.2	7.4	8.3	9.5	10.9	7.0	8.5	9.2	10.1	8.1	9.4	3.3	**1.2**
p-value[a]	0.0116	0.0076	0.0020	0.0117	0.0039	0.0020	0.0020	0.0076	0.0074	0.0020	0.0020	0.0020	0.0039	0.0178	

The best results are highlighted in bold and bracket and the suboptimal results are highlighted in bold.
[a] p-value < 0.05 indicating that the RI of this algorithm is significantly lower than that of DeCITS.
[b] The mean±standard deviation are reported from 5 repeated runs with different random seeds.

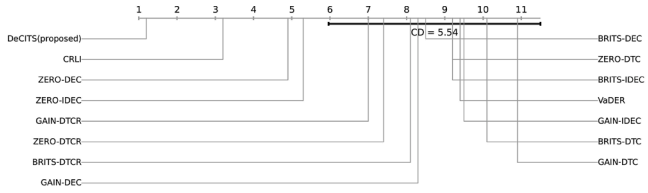
algorithms, the average rank tests were performed on NMI and RI, with the results displayed in the penultimate row of Tables 2 and 3. DeCITS exhibits the lowest average rank score in terms of both NMI and RI, surpassing those of the other algorithms. In addition, the paired Wilcoxon signed-rank tests (Demšar, 2006) were conducted between DeCITS and the other algorithms, and the corresponding p-values were reported in the last row of Tables 2 and 3. All p-values are below 0.05, indicating that DeCITS has a statistically significant advantage across all 10 datasets and can effectively improve NMI and RI.

To further confirm the superiority of DeCITS, Fig. 2 present the critical difference (CD) diagrams based on the Nemenyi test ($\alpha = 0.05$). The horizontal axis represents the average rank of each method across all datasets, where a smaller rank (further left) indicates better overall performance. Methods connected by a bold horizontal line are not significantly different. As shown in Fig. 2, DeCITS achieves the lowest average rank in terms of NMI (1.2) and RI (1.2), and the rank differences between DeCITS and all other methods exceed the critical difference (CD = 5.54), demonstrating that DeCITS is statistically significantly superior to all compared methods.

Moreover, the results for NMI and RI (Tables 2 and 3) demonstrate that DeCITS outperforms the two-stage methods, indicating that jointly optimizing the imputation and clustering processes can enhance the performance of clustering on incomplete time series data. In comparison to the existing one-stage method, VaDER, DeCITS is superior on 9 out of 10 datasets presented in Tables 2 and 3, suggesting that our method effectively mitigates error propagation from imputation to clustering.



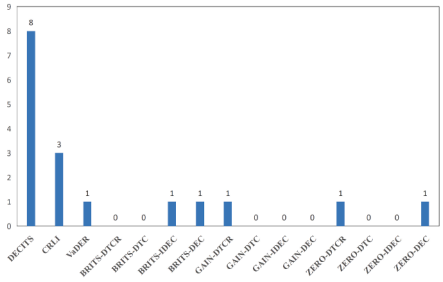
(a) NMI



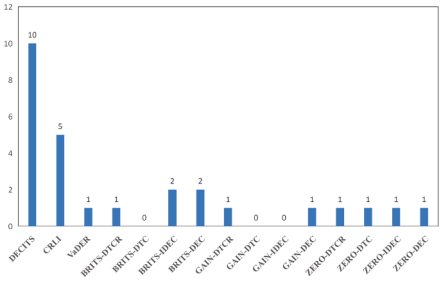
(b) RI

Fig. 2. Critical difference diagram of the average ranks for all compared methods (w.r.t. the results in Tablea 2 and 3).

Additionally, Figs. 3 and 4 visualize the frequency with which different methods are ranked in the top 1 or top 2 positions based on NMI and RI. As shown in these figures, DeCITS consistently ranks in the top 2 across all ten datasets for both NMI and RI, corroborating the highly

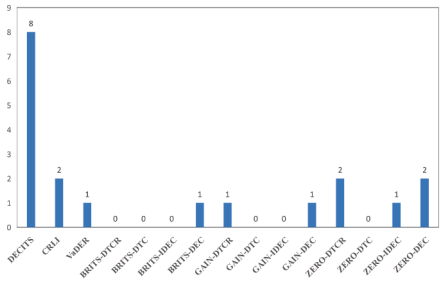


(a) Top-1 count (NMI)

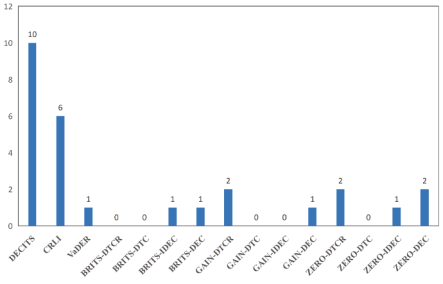


(b) Top-2 count (NMI)

Fig. 3. Frequency of each method ranking in the Top-1 or Top-2 across the ten datasets (w.r.t. the results in Table 2).



(a) Top-1 count (RI)



(b) Top-2 count (RI)

Fig. 4. Frequency of each method ranking in the Top-1 or Top-2 across the ten datasets (w.r.t. the results in Table 3).

competitive clustering performance of the proposed DeCITS method in comparison with state-of-the-art techniques.

4.5. Imputation performance

To directly examine whether DeCITS reduces imputation bias and error propagation, we report the imputation errors and clustering performance of three imputation methods (ZERO, GAIN, and BRITS) and DeCITS on eight UCR datasets with a 20% missing ratio. For a fair NMI comparison, a unified K-means protocol is adopted. For DeCITS, K-means is applied directly to the representations learned at the best epoch. For the three two-stage methods (ZERO, GAIN, and BRITS), K-means is applied to the test sequences after imputation. Specifically, Table 4 summarizes the MAE and RMSE at the masked positions, together with the clustering NMI, for each method under the same experimental setting.

As shown in Table 4, DeCITS achieves the lowest MAE on 5 out of 8 datasets, while obtaining the lowest RMSE on 6 out of 8 datasets, showing that the adversarial strategy generates imputed values that are closer to the ground truth. Even on datasets where DeCITS does not attain the lowest MAE/RMSE, the gap to the leading methods remains marginal. On the Meat dataset, for instance, its imputation values are comparable to those of GAIN and BRITS, and all three methods yield low MAE and RMSE. Consequently, DeCITS still yields the best final clustering performance on this dataset. These results suggest that even when imputation errors are similar, the joint optimization and contrastive learning in DeCITS make the learned representations more robust to the residual imputation noise, thereby effectively mitigating error propagation from imputation to clustering. Notably, on the DiatomSizeReduction dataset, which has the smallest training set, DeCITS achieves the lowest MAE/RMSE but a relatively low NMI. This can be attributed to the strong temporal smoothness of the data, which makes missing values numerically easy to recover, whereas the class distributions overlap substantially in the feature space, resulting in weak cluster separability.

4.6. Ablation study

An ablation study on ten datasets was conducted to gain deeper insights into why DeCITS excels in time series clustering (Tables 5 and 6). The results in Table 5 show that DeCITS achieves the best results on 7 out of 10 datasets. Regarding Table 6, DeCITS secures the top spot on 8 datasets. For single-module ablation, in the absence of the joint optimization strategy, the NMI and RI of the model decrease significantly on almost all datasets, especially on real-world datasets. The absence of the other three modules also leads to varying degrees of performance degradation, although the impact is smaller. Overall, DeCITS with the full architecture achieves the highest average NMI and RI across all datasets, and outperforms the majority of its ablations, underscoring that jointly optimizing the task and integrating contrastive, adversarial, and reconstruction modules are pivotal in enhancing clustering performance. By computing the variance, we observe that DeCITS exhibits the largest variance in NMI. This may be because it leads by a considerable margin on datasets where it performs well, thereby enlarging the performance range across datasets. In contrast, for the RI metric, where the overall values are already high and the room for improvement is limited, the variance of the model ranks third among all models.

It is worth noting that on two real-world incomplete time series datasets (Ali-v3 and Chen) with long sequences and limited samples, the full model underperforms certain ablated variants. This may be attributed to the mismatch between model complexity and data scale. On the one hand, the multi-module joint framework requires sufficient samples to fully exploit its capacity. On the other hand, the additional modules may introduce overfitting or optimization instability on small datasets with long sequences. Removing a specific module could reduce model complexity and act as a form of regularization. This observation suggests that future work could explore adaptive module weighting or small-sample regularization to further improve robustness on such datasets.

Fig. 5 visualizes the distribution of representations learned from DeCITS and models without joint optimization or the three introduced modules. When comparing DeCITS to its four ablations, DeCITS achieves improvements of 0.35, 0.07, 0.10, and 0.08 in NMI for the configurations without jointly optimization, without contrastive learning, without adversarial learning, and without the reconstruction mod-

Table 4
Comparison of MAE, RMSE, and NMI among the four methods across the eight UCR datasets.

DATASETS	MAE				RMSE				NMI			
	ZERO	GAIN	BRITS	Ours	ZERO	GAIN	BRITS	Ours	ZERO	GAIN	BRITS	Ours
Adiac	30.66	11.48	1.71	**0.49**	5.89	2.51	1.21	**0.21**	0.39	0.43	0.49	**0.59**
Beef	66.43	71.60	39.75	**9.92**	9.71	12.50	5.38	**1.64**	0.25	0.30	0.24	**0.46**
BeetleFly	84.25	148.98	**46.88**	61.40	10.00	17.53	**5.95**	7.35	0.16	0.03	0.16	**0.34**
CBF	21.45	23.54	14.33	**10.53**	5.01	5.91	3.59	**2.73**	0.35	0.35	0.36	**0.68**
CricketX	44.69	98.65	**11.52**	12.09	7.72	16.20	3.37	**3.32**	0.26	0.24	0.27	**0.39**
CricketY	45.07	65.17	14.06	**11.61**	7.76	11.38	4.04	**2.82**	0.32	0.33	0.32	**0.42**
DiatomSizeReduction	60.56	9.56	42.46	**4.82**	8.31	1.92	6.71	**0.69**	0.73	0.63	**0.75**	0.61
Meat	57.47	**2.09**	4.30	9.22	9.16	**0.31**	0.78	1.22	0.01	0.65	0.61	**0.66**

The best results are highlighted in bold.

Table 5
Ablation analysis in DeCITS (w.r.t.NMI).

Joint Optimization		✓	×	✓	✓	✓
Contrastive Module		✓	✓	×	✓	✓
Adversarial Module		✓	✓	✓	×	✓
Reconstruction Module		✓	✓	✓	✓	×
Datasets	Ali-v1	**0.68**	0.22	**0.68**	**0.68**	**0.68**
	Ali-v3	0.79	0.37	**0.81**	**0.81**	0.80
	BloodSample	**0.57**	0.22	0.50	0.47	0.49
	Chen	0.39	0.00	0.35	**0.46**	0.40
	Vote	**0.80**	0.37	0.62	0.60	0.66
	Physionet	**0.03**	0.00	**0.03**	0.02	0.01
	Adiac	0.59	0.58	0.53	0.43	**0.60**
	Beef	**0.53**	0.42	0.48	0.46	0.46
	BeetleFly	**0.35**	**0.35**	**0.35**	**0.35**	**0.35**
	CBF	**0.74**	0.65	0.58	0.41	0.26
AVG NMI		0.55	0.32	0.47	0.47	0.49
Variance		0.0568	0.0460	0.0532	0.0439	0.0462

Table 6
Ablation analysis in DeCITS (w.r.t.RI).

Joint Optimization		✓	×	✓	✓	✓
Contrastive Module		✓	✓	×	✓	✓
Adversarial Module		✓	✓	✓	×	✓
Reconstruction Module		✓	✓	✓	✓	×
Datasets	Ali-v1	**0.85**	0.62	**0.85**	**0.85**	**0.85**
	Ali-v3	0.88	0.65	**0.91**	**0.91**	0.90
	BloodSample	**0.86**	0.67	0.83	0.82	0.83
	Chen	0.74	0.50	0.69	**0.77**	0.69
	Vote	**0.93**	0.72	0.82	0.81	0.84
	Physionet	**0.76**	**0.76**	**0.76**	**0.76**	**0.76**
	Adiac	**0.95**	**0.95**	0.94	**0.95**	**0.95**
	Beef	**0.79**	0.73	0.75	0.75	0.75
	BeetleFly	**0.61**	**0.61**	**0.61**	**0.61**	**0.61**
	CBF	**0.86**	0.83	0.76	0.74	0.62
AVG RI		0.82	0.70	0.78	0.79	0.79
Variance		0.0102	0.0155	0.0131	0.0092	0.0099

ules, respectively. Fig. 5 shows that the representation distribution of the model exhibits increasingly distinct clustering structures as NMI increases, further illustrating the effectiveness of these four components within the proposed DeCITS model.

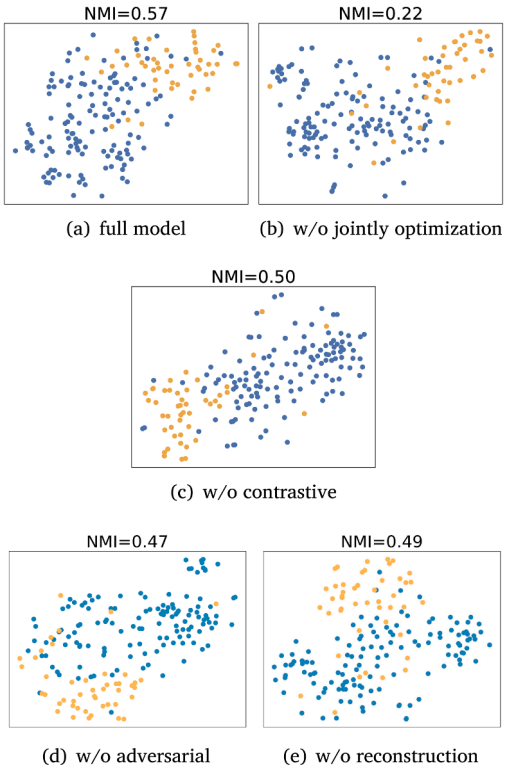
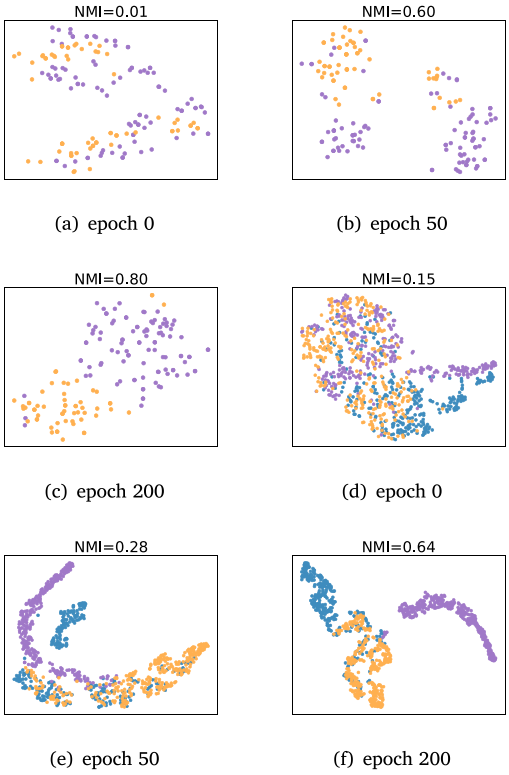


Fig. 5. Visualizations of experiments on the original BloodSample dataset. (a) DeCITS, (b) DeCITS without jointly optimization, (c) DeCITS without contrastive module, (d) DeCITS without adversarial module, (e) DeCITS without reconstruction module.

4.7. Visualization analysis

In this section, we visualize the learned feature representations obtained through Eqs. (15) and (16) during the training process. Specifically, we utilize t-SNE (Van der Maaten & Hinton, 2008) to project the feature representations into a two-dimensional (2-D) space for visualization. These experiments are conducted on ten datasets, demonstrating the beneficial impact of the contrastive learning module on the clustering task. For the visualization, we focus on the HouseVote and CBF datasets.

As illustrated in Fig. 6, the parameters of the model are initially randomized, resulting in scattered and chaotic feature representations that lead to poor clustering outcomes. By Epoch 50, the feature representations begin to show increased separability, with reduced intra-cluster distances and expanded inter-cluster distances. By Epoch 200, a stable, well-learned feature representation is evident as the model converges.

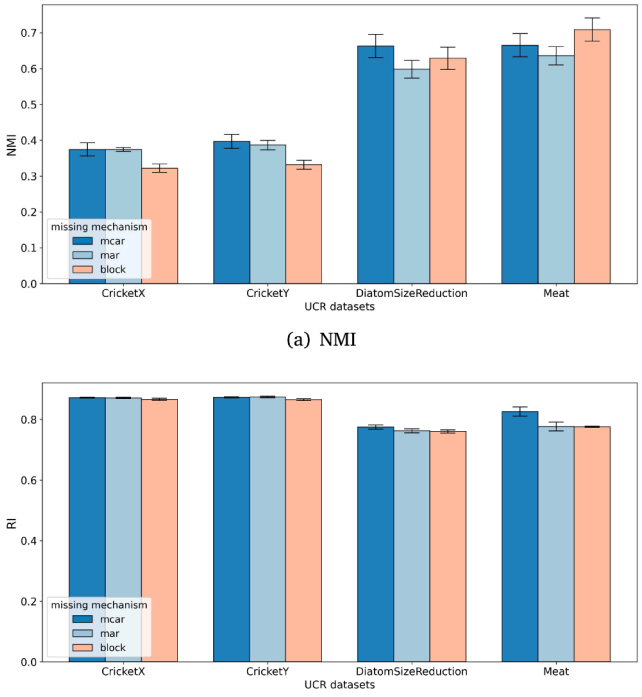


(a) epoch 0 (b) epoch 50

(c) epoch 200 (d) epoch 0

(e) epoch 50 (f) epoch 200

Fig. 6. Visualization with t-SNE on the two datasets. (a-c) HouseVote (d-f) CBF.



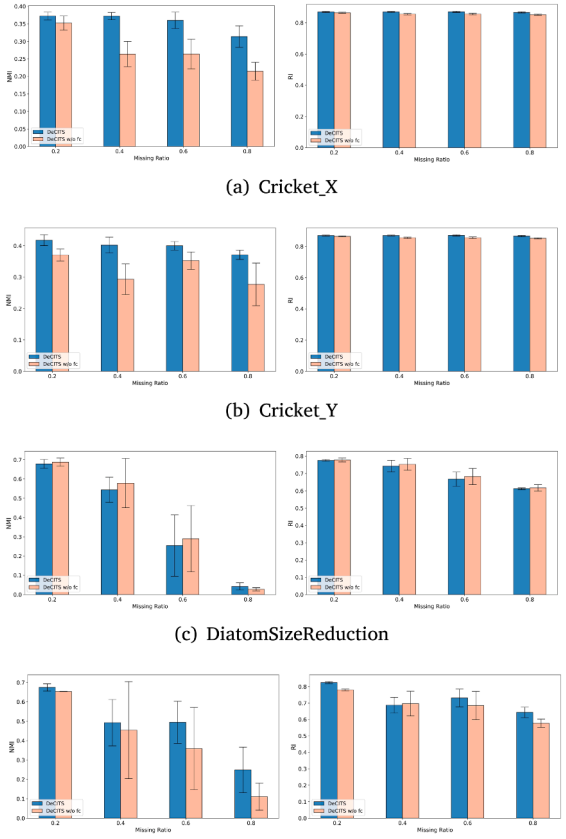
(a) NMI

(b) RI

Fig. 7. NMI and RI (mean±std) of four UCR datasets over 5 runs under different missing mechanisms.

4.8. Stability test

To evaluate model stability on different missing mechanisms, we conducted five repeated training runs under three missingness mechanisms (MAR, MCAR, and block missing) using different random seeds on four datasets: Cricket X, Cricket Y, DiatomSizeReduction, and Meat.



(a) Cricket_X

(b) Cricket_Y

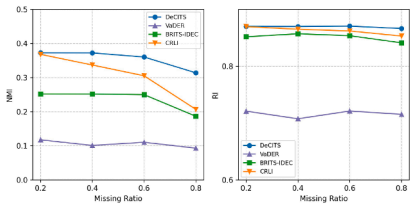
(c) DiatomSizeReduction

(d) Meat

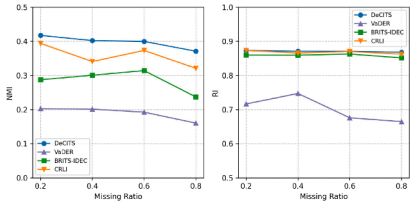
Fig. 8. Ablation experiments on the fully connected layer across four UCR datasets under different missing ratios.

We then report the mean±standard deviation for NMI and RI. As illustrated in Fig. 7, the overall clustering performance under MCAR is better than that under MAR and block missing. This is because MCAR merely causes a random reduction in information without distorting the data distribution, MAR introduces conditional bias, and block missing disrupts the temporal structure and leads to substantial information loss. However, the DeCITS model exhibits relatively stable overall performance across the three missing mechanisms, with NMI and RI values fluctuating within a small range, particularly for RI. Furthermore, based on the coefficient of variation (standard deviation / mean), none of the four models exceeds 5% for either NMI or RI. This result further demonstrates that the proposed DeCITS is highly robust, achieving stable clustering performance on datasets from different domains under various missingness mechanisms.

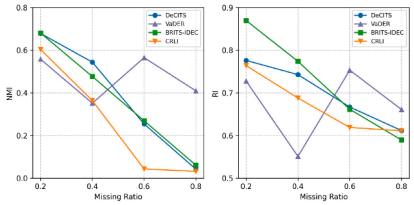
In addition, to demonstrate the stabilizing effect of the fully connected layer in the imputation and clustering processes, we conducted ablation experiments on the fully connected layer under different missing ratios. For each experiment, we conducted five independent runs with different random seeds and reported the results as the mean±standard deviation in a bar chart with error bars. Fig. 8 compares the NMI and RI of DeCITS with and without the fully connected layer at missing ratios of 20%, 40%, 60%, and 80%. The results show that at a missing ratio of 20%, the performance with and without the fully connected layer is comparable. As the missing ratio increases, DeCITS without the fully connected layer achieves lower mean NMI and RI than DeCITS with the fully connected layer. In addition, the results of repeated training exhibit greater fluctuations as the missing ratio increases, and the model without the fully connected layer shows even larger fluctuations and is less stable. This indicates that inserting a fully connected layer to decou-



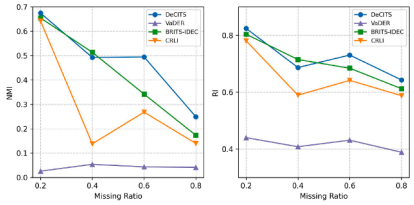
(a) Cricket_X



(b) Cricket_Y



(c) DiatomSizeReduction



(d) Meat

Fig. 9. Average performance of different methods over 5 runs on four UCR datasets under different missing ratios.

ple imputation and clustering is effective, and jointly optimizing these two tasks mitigates model instability.

4.9. Sensitivity analysis under different missing ratios

To evaluate the sensitivity of different algorithms to missing ratios, we conducted repeated experiments on four UCR datasets with four algorithms under different missing ratios (20%, 40%, 60%, and 80%), and plotted line charts of NMI and RI using the mean values. As shown in the line charts (Fig. 9), almost all algorithms achieve the best clustering performance at a missing ratio of 20%. As the missing ratio increases, the clustering performance gradually declines. On the CricketX and CricketY datasets, the NMI and RI curves of DeCITS consistently remain above those of the other three algorithms, and a significant drop in NMI and RI occurs only when the missing ratio reaches 80%, demonstrating strong robustness.

On the DiatomSizeReduction and Meat datasets, the performance of the algorithms degrades more noticeably. Nevertheless, DeCITS still maintains a more stable average performance on the Meat dataset. It is worth noting that on datasets with more limited time series samples, such as DiatomSizeReduction, VaDER outperforms the other three algorithms when the missing ratio reaches 60% and 80%, respectively. This may be attributed to the fact that VaDER has a simpler architecture with fewer parameters, making it more suitable for small datasets. In contrast, the multi-module joint framework of DeCITS requires more sufficient data samples. Specifically, contrastive learning and adversarial training require sufficient samples to learn stable representations, while the joint optimization of multiple objectives is more prone to overfitting or local optima in small-sample regimes. This observation suggests that future work could improve the robustness of DeCITS on small datasets through small-sample regularization, data augmentation, or transfer learning.

4.10. Limitations

Although DeCITS achieves promising results on the evaluated benchmarks, it should be acknowledged that the current experimental datasets are relatively small, with no dataset exceeding ten thousand samples, partly due to privacy or proprietary restrictions on large-scale datasets and limited computational resources. This limitation may restrict the generalizability of the conclusions to larger-scale real-world applications. Theoretically, DeCITS has an $O(NT)$ per-epoch complexity, linear in both sample number N and sequence length T. The bidirectional GRU encoder–decoder and the adversarial and contrastive losses support mini-batch SGD, while the clustering loss scales as $O(NK_c)$. These properties suggest that DeCITS can scale to larger datasets without a significant increase in computational cost. In future work, DeCITS will be evaluated on larger-scale datasets from domains such as healthcare, sensor networks, and finance. Distributed training and efficient sampling strategies will also be explored to further improve scalability. Overall, the linear complexity and modular design of DeCITS provide a solid foundation for such extensions.

Another limitation of this work is that it focuses on static time series, assuming that all data are available before training. In real-world applications, however, data are often collected continuously and dynamically, with new observations arriving over time. This streaming data setting introduces additional challenges, such as catastrophic forgetting, storage constraints, and the need to adapt to new patterns without revisiting historical data (Wan et al., 2025). Recent advances in continual multi-view clustering, such as FCMVC-IV (Wan et al., 2024) and CMVC (Wan et al., 2022), have addressed sequentially arriving and incomplete views. Although these methods target multi-view data rather than feature-level missingness in time series, their continual learning and incremental update mechanisms offer valuable insights. Extending DeCITS to a continual setting, where incomplete time series arrive sequentially, remains an open direction for future work. This would require addressing memory-efficient representation learning, adaptation to evolving missingness patterns, and online clustering with limited historical data.

5. Conclusions

In summary, this study introduces a contrastive learning task for handling incomplete time series data and jointly optimizes the imputation and clustering processes from two perspectives. Additionally, an adversarial strategy is implemented to mitigate error propagation from the imputation to clustering stages. Experimental results demonstrate that DeCITS surpasses both existing two-stage and one-stage methods in incomplete time series clustering. The ablation study and stability test validate the effectiveness of the introduced modules and the stability of the model. Furthermore, visualization and sensitivity analysis highlight the efficacy of the learned representations and identifies the optimal missing ratio.

In this study, the proposed DeCITS algorithm achieves relatively favorable clustering performance on datasets from multiple domains, demonstrating its excellent generalization ability. When time series from different real-world domains contain a certain amount of missing data, this algorithm can perform superior imputation and thereby reduce the negative impact of missing data on downstream clustering tasks. Meanwhile, the joint optimization strategy improves the effectiveness of the learning process, which is beneficial for the practical implementation

and deployment of the algorithm. This strategy provides a promising direction for the future development of incomplete time series analysis and establishes a solid foundation for building more robust and efficient clustering frameworks.

CRediT authorship contribution statement

Ying Zhong: Writing – original draft, Visualization, Software, Methodology, Formal analysis, Data curation, Conceptualization; **Haiyan Wang:** Writing – review & editing, Validation, Investigation, Funding acquisition, Conceptualization; **Dong Huang:** Writing – review & editing, Methodology, Investigation, Funding acquisition, Conceptualization; **Yuan-Kun Xu:** Writing – original draft, Visualization, Validation, Software, Methodology; **Chang-Dong Wang:** Writing – review & editing, Resources, Project administration, Investigation; **Huiru Zhou:** Writing – review & editing, Writing – original draft, Visualization, Validation, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization.

Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 32402464 and Grant No. 62202176), the Natural Science Foundation of Guangdong Province (Grant No. 2023A1515012885), and the Guangzhou Basic and Applied Basic Research Project (Grant No. 2025A04J3924).

References

Alahamade, W., Lake, I., Reeves, C. E., & De La Iglesia, B. (2022). A multi-variate time series clustering approach based on intermediate fusion: A case study in air pollution data imputation. *Neurocomputing*, *490*, 229–245. https://doi.org/10.1016/j.neucom.2021.09.079

Alizadeh, A. A., Eisen, M. B., Davis, R. E., Ma, C., Lossos, I. S., Rosenwald, A., Boldrick, J. C., Sabet, H., Tran, T., Yu, X., Powell, J. I., Yang, L., Marti, G. E., Moore, T., Jr, J. H., Lu, L., Lewis, D. B., Tibshirani, R., Sherlock, R. , Chan, W. C., Greiner, T. C., Weisenburger, D. D., Armitage, J. O., Warnke, R., Levy, R., Wilson, W., Grever, M. R., Byrd, J. C., Botstein, D., Brown, P. O., & Staudt, L. M. (2000). Distinct types of diffuse large b-cell lymphoma identified by gene expression profiling. *Nature*, *403*(6769), 503–511. https://doi.org/10.1038/35000501

Bar, N., Nikparvar, B., Jayavelu, N. D., & Roessler, F. K. (2022). Constrained fourier estimation of short-term time-series gene expression data reduces noise and improves clustering and gene regulatory network predictions. *BMC Bioinformatics*, *23*(1). https://doi.org/10.1186/s12859-022-04839-z

Barazandeh, I., Haratizadeh, S., & Sermpinis, G. (2025). A temporal graph-based contrastive approach for financial time series forecasting. *Engineering Applications of Artificial Intelligence*, *153*. https://doi.org/10.1016/j.engappai.2025.110834

Bianchi, F. M., Mikalsen, K. O., & Jenssen, R. (2017). Learning compressed representations of blood samples time series with missing data. *arXiv e-prints*, (p. arXiv:1710.07547). https://doi.org/10.48550/arXiv.1710.07547

Cao, W., Wang, D., Li, J., Zhou, H., Li, L., & Li, Y. (2018). BRITS: Bidirectional recurrent imputation for time series. In *Proceedings of 32nd conference on neural information processing systems (neurIPS 2018)*. Montréal, Canada.

Cheema, J. R. (2014). A review of missing data handling methods in education research. *Review of Educational Research*, *84*(4), 487–508. https://doi.org/10.3102/0034654314532697

Chen, B., Zheng, H., Wang, L., Hellwich, O., Chen, C., Yang, L., Liu, T., Luo, G., Bao, A., & Chen, X. (2022). A joint learning im-biLSTM model for incomplete time-series sentinel-2a data imputation and crop classification. *International Journal of Applied Earth Observation and Geoinformation*, *108*, 102762. https://doi.org/10.1016/j.jag.2022.102762

Chen, X., Cheung, S. T., So, S., Fan, S. T., Barry, C., Higgins, J., Lai, K.-M., Ji, J., Dudoit, S., Ng, I. O. L. et al. (2002). Gene expression patterns in human liver cancers. *Molecular Biology of the Cell*, *13*(6), 1929–1939. https://doi.org/10.1091/mbc.02-02-0023

Chen, X., & He, K. (2021). Exploring simple siamese representation learning. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)* (pp. 15750–15758). Nashville, USA.

Cho, K., Van Merrienboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. arXiv:1406.1078). https://doi.org/10.48550/arXiv.1406.1078

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *The Journal of Machine Learning Research*, *7*, 1–30.

Deng, J., Chan, P. P. K., & Yeung, D. S. (2025). Real-world nighttime image dehazing using contrastive and adversarial learning. *Pattern Recognition*, *165*. https://doi.org/10.1016/j.patcog.2025.111596

Du, W., Cote, D., & Liu, Y. (2023). Saits: Self-attention-based imputation for time series. *Expert Systems with Application*, *219*. https://doi.org/10.1016/j.eswa.2023.119619

Eldele, E., Ragab, M., Chen, Z., Wu, M., Kwoh, C.-K., Li, X., & Guan, C. (2023). Self-supervised contrastive representation learning for semi-supervised time-series classification. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *45*(12), 15604–15618. https://doi.org/10.1109/TPAMI.2023.3308189

Fortuin, V., Baranchuk, D., Raetsch, G., & Mandt, S. (2020). Gp-vae: Deep probabilistic time series imputation. In S. Chiappa, & R. Calandra (Eds.), *Proceedings of the twenty third international conference on artificial intelligence and statistics* (pp. 1651–1661). PMLR (*volume. 108*). Proceedings of Machine Learning Research. https://proceedings.mlr.press/v108/fortuin20a.html.

Gao, P., Yang, X., Huang, K., Zhang, R., & Goulermas, J. Y. (2023). Explainable tensorized neural ordinary differential equations for arbitrary-step time series prediction. *IEEE Transactions on Knowledge and Data Engineering*, *35*(6), 5837–5850. https://doi.org/10.1109/TKDE.2022.3167536

Guo, W., Zhang, Z., Meng, Y., Zhang, W., Gao, S., & Tang, P. (2025). Deep temporal joint clustering for satellite image time-series analysis. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, *18*, 1272–1287. https://doi.org/10.1109/JSTARS.2024.3502247

Guo, X., Gao, L., Liu, X., & Yin, J. (2017). Improved deep embedded clustering with local structure preservation. In *Proceedings of international joint conferences on artificial intelligence (IJCAI)* (pp. 1753–1759). Melbourne, Australia.

Hao, S., Wang, Z., Alexander, A. D., Yuan, J., & Zhang, W. (2023). MICOS: Mixed supervised contrastive learning for multivariate time series classification. *Knowledge-Based Systems*, *260*, 110158. https://doi.org/10.1016/j.knosys.2022.110158

Hu, J., Hu, Z., Li, T., & Du, S. (2023). A contrastive learning based universal representation for time series forecasting. *Information Sciences*, *635*, 86–98. https://doi.org/10.1016/j.ins.2023.03.143

Huang, F., & Deng, Y. (2023). Tcgan: Convolutional generative adversarial network for time series classification and clustering. *Neural Networks*, *165*, 868–883. https://doi.org/10.1016/j.neunet.2023.06.033

Hussein, D., Belkhouja, T., Bhat, G., & Doppa, J. (2025). Sensor-aware data imputation for time-series machine learning on low-power wearable devices. *ACM Transactions on Design Automation of Electronic Systems*, *30*(1). https://doi.org/10.1145/3698195

de Jong, J., Emon, M. A., Wu, P., Karki, R., Sood, M., Godard, P., Ahmad, A., Vrooman, H., Hofmann-Apitius, M., & Fröhlich, H. (2019). Deep learning for clustering of multivariate clinical patient trajectories with missing values. *GigaScience*, *8*, 1–14. https://doi.org/10.1093/gigascience/giz134

Kashinath, K., Mustafa, M., Albert, A., Wu, J. L., Jiang, C., Esmaeilzadeh, S., Azizzadenesheli, K., Wang, R., Chattopadhyay, A., Singh, A. et al. (2021). Physics-informed machine learning: Case studies for weather and climate modelling. *Philosophical Transactions of the Royal Society A*, *379*(2194), 20200093. https://doi.org/10.1098/rsta.2020.0093

Lai, X., Zhang, Z., Zhang, L., Lu, W., & Li, Z. (2025). Dynamic graph-based bilateral recurrent imputation network for multivariate time series. *Neural Networks*, *186*, 107298. https://doi.org/10.1016/j.neunet.2025.107298

Li, Y., Chen, Z., Wen, Z., Xiao, X., & Zhu, M. (2026). Time-frequency contrastive learning with context modeling for time series anomaly prediction. *Neural Networks*, *197*, 108494. https://doi.org/10.1016/j.neunet.2025.108494

Li, Y., Du, M., Zhang, W., Jiang, X., & Dong, Y. (2024). Feature weighting-based deep fuzzy c-means for clustering incomplete time series. *IEEE Transactions on Fuzzy Systems*, *32*(12), 6835–6847. https://doi.org/10.1109/TFUZZ.2024.3466175

Liao, T. W. (2005). Clustering of time series data - a survey. *Pattern Recognition*, *38*(11), 1857–1874. https://doi.org/10.1016/j.patcog.2005.01.025

Liu, S., Chen, J., He, S., Shi, Z., & Zhou, Z. (2023). Few-shot learning under domain shift: Attentional contrastive calibrated transformer of time series for fault diagnosis under sharp speed variation. *Mechanical Systems and Signal Processing*, *189*, 110071. https://doi.org/10.1016/j.ymssp.2022.110071

Luo, Y., Cai, X., Zhang, Y., Xu, J., & Yuan, X. (2018). Multivariate time series imputation with generative adversarial networks. *Advances in Neural Information Processing Systems (NeurIPS)*, *31*.

Ma, Q., Chen, C., Li, S., & Cottrell, G. W. (2021). Learning representations for incomplete time series clustering. In *Proceedings of the AAAI conference on artificial intelligence (AAAI)* (pp. 8837–8846). Vancouver, Canada (*volume. 35*).

Ma, Q., Zheng, J., Li, S., & Cottrell, G. W. (2019). Learning representations for time series clustering. In *Proceedings of 33rd conference on neural information processing systems (neurIPS 2019)*. Vancouver, Canada (*volume. 32*).

Mejri, N., Lopez-Fuentes, L., Roy, K., Chernakov, P., Ghorbel, E., & Aouada, D. (2024). Unsupervised anomaly detection in time-series: An extensive evaluation and analysis of state-of-the-art methods. *Expert Systems with Application*, *256*. https://doi.org/10.1016/j.eswa.2024.124922

Øyvind Mikalsen, K., Bianchi, F. M., Soguero-Ruiz, C., & Jenssen, R. (2018). Time series cluster kernel for learning similarities between multivariate time series with missing data. *Pattern Recognition*, *76*, 569–581. https://www.sciencedirect.com/science/article/pii/S0031320317304843. https://doi.org/10.1016/j.patcog.2017.11.030

Peng, W., Lin, Y., Guo, S., Tang, W., Liu, L., & Wan, H. (2023). Generative-contrastive-attentive spatial-temporal network for traffic data imputation. In *Proceedings of pacific-asia conference on advances in knowledge discovery and data mining (KDD)* (pp. 45–56). Springer (*volume. 13938*).

Rand, W. M. (1971). Objective criteria for the evaluation of clustering methods. *Journal of the American Statistical Association*, *66*(336), 846–850. https://doi.org/10.1080/01621459.1971.10482356

Sai Madiraju, N., Sadat, S. M., Fisher, D., & Karimabadi, H. (2018). Deep temporal clustering: Fully unsupervised learning of time-domain features. *arXiv e-prints*, (p. arXiv: 1802.01059). https://doi.org/10.48550/arXiv.1802.01059

Silva, I., Moody, G., Scott, D. J., Celi, L. A., & Mark, R. G. (2012). Predicting in-hospital mortality of icu patients: The physionet/computing in cardiology challenge 2012. In *2012 Computing in cardiology* (pp. 245–248). IEEE.

Strehl, A., & Ghosh, J. (2002). Cluster ensembles—a knowledge reuse framework for combining multiple partitions. *The Journal of Machine Learning Research*, *3*(Dec), 583–617.

Sun, X., Luo, Y., Wang, Z., & Chen, A. (2026). Augmented graph contrastive learning with view separation. *Neurocomputing*, *665*, 132156. https://doi.org/10.1016/j.neucom.2025.132156

Tan, Z., Luo, X., Liu, Y., & Zhang, Y. (2026). Mask the redundancy: Evolving masking representation learning for multivariate time-series clustering. In *Proceedings of the fortieth AAAI conference on artificial intelligence and thirty-eighth conference on innovative applications of artificial intelligence and sixteenth symposium on educational advances in artificial intelligence* AAAI'26/IAAI'26/EAAI'26. AAAI Press. https://doi.org/10.1609/aaai.v40i30.39777

Van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *The Journal of Machine Learning Research*, *9*, 2579–2605.

Wan, X., Liu, J., Liang, W., Liu, X., Wen, Y., & Zhu, E. (2022). Continual multi-view clustering. In *Proceedings of the 30th ACM international conference on multimedia* MM '22 (p. 3676–3684). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3503161.3547864. https://doi.org/10.1145/3503161.3547864

Wan, X., Liu, J., Yu, H., Qu, Q., Li, A., Liu, X., Liang, K., Dong, Z., & Zhu, E. (2025). Contrastive continual multiview clustering with filtered structural fusion. *IEEE Transactions on Neural Networks and Learning Systems*, *36*(8), 14829–14842. https://doi.org/10.1109/TNNLS.2024.3502455

Wan, X., Xiao, B., Liu, X., Liu, J., Liang, W., & Zhu, E. (2024). Fast continual multi-view clustering with incomplete views. *IEEE Transactions on Image Processing*, *33*, 2995–3008. https://doi.org/10.1109/TIP.2024.3388974

Wu, R., & Keogh, E. J. (2023). Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress. *IEEE Transactions on Knowledge and Data Engineering*, *35*(3), 2421–2429. https://doi.org/10.1109/TKDE.2021.3112126

Xie, J., Girshick, R., & Farhadi, A. (2016). Unsupervised deep embedding for clustering analysis. In *Proceedings of the 33rd international conference on machine learning (ICML)* (pp. 478–487). New York, USA: PMLR (*volume. 48*).

Yoon, J., Jordon, J., & Schaar, M. (2018). Gain: Missing data imputation using generative adversarial nets. In *Proceedings of international conference on machine learning(ICML)* (pp. 5689–5698). Stockholm Sweden: PMLR.

Zhang, S., Wang, J., Nie, S., Yue, J., Zhu, W., & Lin, Y. (2025). Loss or gain: Hierarchical conditional information bottleneck approach for incomplete time series classification. In *Proceedings of the 31st ACM SIGKDD conference on knowledge discovery and data mining v.2* KDD '25 (p. 3796–3807). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3711896.3737033

Zhang, Z., Zhang, Y., Zeng, A., Pan, D., Ji, Y., Zhang, Z., & Lin], J. (2023). Time-series data imputation via realistic masking-guided tri-attention bi-GRU. In K. Gal, A. Nowé, G. J. Nalepa, R. Fairstein, & R. Rădulescu (Eds.), *Proceedings of the 26th european conference on artificial intelligence* (pp. 3074–3082). SAGE Publications. https://journals.sagepub.com/doi/abs/10.3233/FAIA230625. https://doi.org/10.3233/FAIA230625

Zhong, Y., Huang, D., & Wang, C.-D. (2022). Deep temporal contrastive clustering. *Neural Processing Letters*, *55*(6), 7869–7885. https://doi.org/10.1007/s11063-023-11287-0

Zhu, Y., Mueen, A., & Keogh, E. (2021). Matrix profile IX: Admissible time series motif discovery with missing data. *IEEE Transactions on Knowledge and Data Engineering*, *33*(6), 2616–2626. https://doi.org/10.1109/TKDE.2019.2950623